InfraRL: Reinforcement Learning for Use-Case-Driven AI Infrastructure Recommendation

MSAI/MSML 635: Reinforcement Learning | Project Proposal

Team: Anupama, Jay, Vishnu

1. Background

When a company wants to deploy an LLM-based application, they face a hard question: what hardware should they run it on? There are GPUs ranging from A10 to H100, models from 3B to 70B parameters, quantization options (INT4 through FP32), and different batching and scaling strategies. Getting this wrong is expensive: either you overspend on hardware you dont need, or you underprovision and miss your latency targets.

Right now most companies figure this out through trial and error, or by talking to cloud sales engineers. Providers like Nebius, AWS, and Azure have dozens of GPU options but no system that looks at your specific use case and tells you what to pick. Companies typically waste 40-60% of their compute budget because of this.

We think this is a good fit for reinforcement learning. You can frame it as a sequential decision problem: the customer profile is the state, the hardware/model/config choices are actions, and the reward is some combination of quality, cost, and SLA compliance.

2. Significance and Problem Statement

The problem we're trying to solve: a customer comes to us with their AI use case (what kind of app, how many queries/day, latency requirements, budget, data sensitivity) and we want to automatically recommend a full infrastructure stack, from which GPUs to use, all the way down to quantization and scaling config. It should keep adjusting as their workload changes over time.

Why we think this is worth working on:

- AI cloud infrastructure is a huge market (projected >$300B by 2027).
- Most teams don't have the expertise to make these decisions. Manual planning takes days/weeks.
- Even if you get it right initially, workloads change. A static recommendation goes stale.
- We looked at related work and couldn't find anyone using RL for this specific problem. RouteLLM and Lodestar pick which model but don't touch hardware or precision. GPU allocation papers work at cluster level, not per-customer.

Novelty: existing work either routes models (RouteLLM, Lodestar) or allocates GPUs at cluster level. Nobody jointly optimizes hardware + model + precision + scaling per customer using RL.

3. Approach

3.1 MDP Formulation

We're framing this as a Markov Decision Process:

State: a vector encoding the customer's profile: use case type, query volume, latency SLA, budget, data sensitivity, peak traffic pattern, etc.

Actions: the agent makes decisions in sequence across 5 layers: (1) architecture type, (2) hardware/tier, (3) model/tier, (4) config/tier (precision, batch size), (5) scaling policy.

Reward: weighted combination: quality score (0.4), budget compliance (0.3), SLA compliance (0.2), and cost efficiency bonus (0.1). The simulator evaluates each action and returns the reward signal that updates the policy.

Constraints: invalid combos like running a 70B model on an A10 (only 24GB memory) get masked out so the agent can't pick them.

3.2 Simulator

Since we can't benchmark real GPUs for every experiment, we're building a calibrated simulator using published performance data. It takes a (hardware, model, config) combo and a customer profile, and outputs estimated quality, latency, and cost. We plan to calibrate from Nebius Token Factory specs, NVIDIA MLPerf, LMSYS Chatbot Arena, and HumanEval.

3.3 RL Algorithms (mapped to course topics)

We want to compare multiple RL approaches from the course:

Method	Course Topic	Role
Epsilon-greedy bandit	K-Armed Bandits	Baseline
Tabular Q-Learning	TD / Q-Learning	Mid-complexity
DQN (3-layer MLP)	Function Approx	Primary agent
PPO	Policy Gradient	Comparison

3.4 Online Continuous Optimization

The initial recommendation isn't the end. After deployment, a contextual bandit watches real workload metrics (actual traffic, observed latency, cost) and suggests adjustments weekly. This handles the fact that workloads aren't static: traffic patterns shift, new models come out, pricing changes.

4. Team Responsibilities

Member	Role	Deliverables
Jay	Infra & Simulation	Simulator, profiles, Gym env, system design
Vishnu & Anupama	RL & Evaluation	Bandit, Q-learning, DQN, PPO, experiments

Timeline:

Weeks	Jay (Infra)	Vishnu & Anupama (RL + Eval)
1-2	Build simulator, Gym env	Bandit, Q-learning baselines
3-4	Refine simulator	DQN implementation + training
5-6	Online sim, ctx bandit	PPO, ablations, Pareto plots
7-8	Demo dashboard, docs	Final eval, report, slides

5. Metrics for Validation

We generate 1000 random customer profiles across 6 use case types (chatbot, code gen, RAG, multimodal, fine-tuning, batch processing). For each profile, the RL agent recommends a stack, and we measure how good that recommendation is.

How we measure 'good':

- Quality: does the recommended model actually perform well on this use case? (target > 0.85 accuracy)
- Budget: did the recommendation stay within the customer's monthly budget? (target > 90% compliance)
- Latency: does the recommended hardware meet the latency SLA? (target > 90% compliance)
- Efficiency: how much of the budget was saved while still meeting quality + SLA?

We compare our RL agent against baselines: random selection, hand-written heuristics, and simpler RL methods (bandit, Q-learning). The key result is a Pareto frontier plot showing quality vs cost, where the RL agent should dominate the baselines.

Ablation studies to prove each component matters:

- Remove hardware-awareness: does cost go up?
- Remove precision tuning: does quality drop?
- Remove scaling policy: do SLA violations spike during peaks?

6. System Architecture

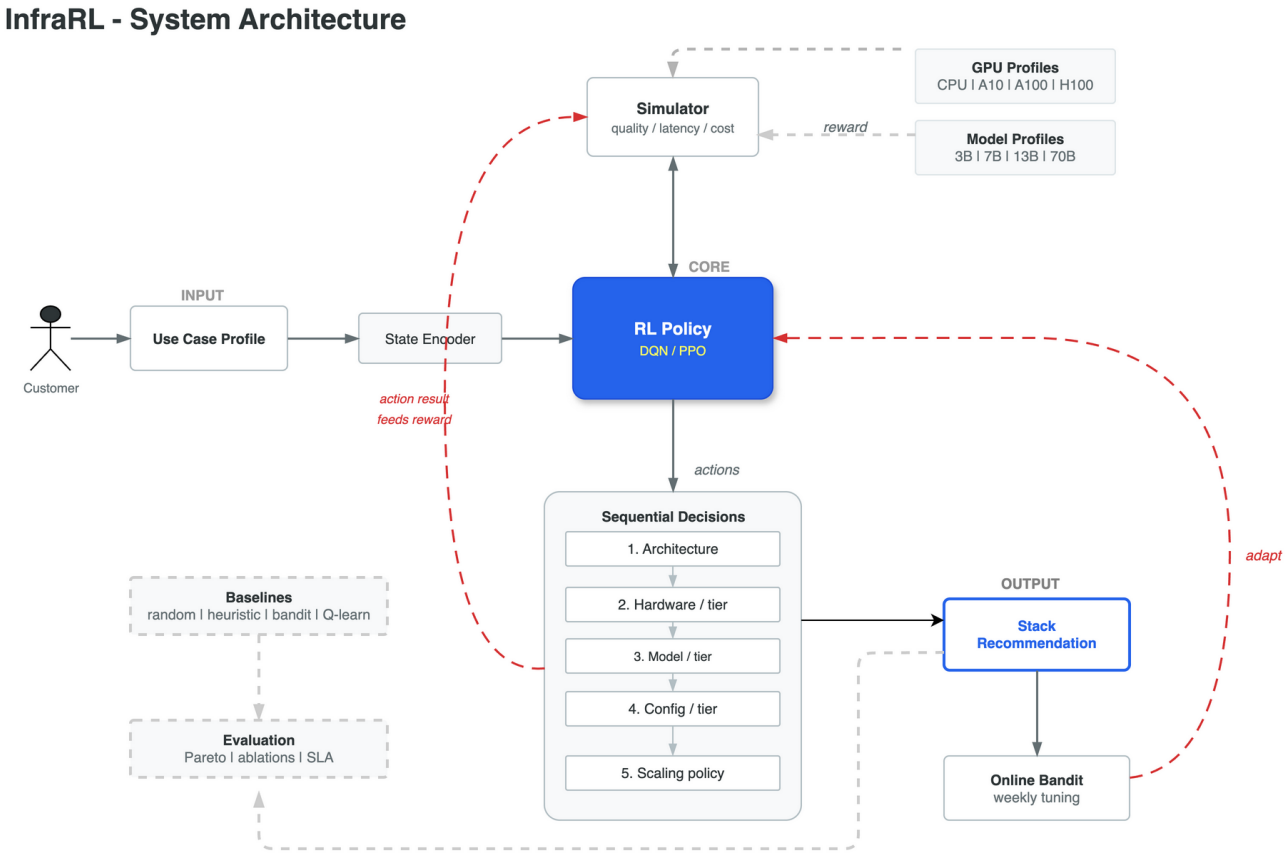


Figure 1: InfraRL system architecture. Dashed red lines show the reward/adaptation loop.

6.1 Expected Results

We expect the RL agent to learn use-case-specific strategies:

- Chatbot: mid-range GPU (A10/A100) + 13B model (balances cost and quality)
- Code generation: H100 + 70B (accuracy over cost)
- Batch processing: CPU or cheap GPU + INT4 (throughput over latency)
- RAG: multi-tier (cheap retrieval on CPU, expensive generation on GPU)

7. Framework and Tools

Gymnasium for RL environment, Stable-Baselines3 for DQN/PPO, custom NumPy for bandit/Q-learning, Matplotlib for plots, Streamlit for demo. We adapt Stable-Baselines3's DQN and PPO implementations [1] for our custom Gymnasium environment, modifying the reward function and action masking for our infrastructure recommendation domain. RL policy training is lightweight (small MLP). Compute resources via UMD Zaratan HPC cluster.

References

[1] Raffin et al. Stable-Baselines3. JMLR, 2021.
[2] Sutton & Barto. RL: An Introduction. 2nd ed., MIT Press, 2018.
[3] Nebius. Token Factory. nebius.com, 2026.
[4] Ong et al. RouteLLM. ICLR, 2025.
[5] Schulman et al. PPO. arXiv:1707.06347, 2017.
[6] Mnih et al. DQN. Nature, 2015.
[7] Chen et al. Lodestar. arXiv:2606.00946, 2026.
[8] Zhang et al. GPU Allocation via RL. arXiv:2507.10259, 2026.